TEMA 5 – Fragments

> **Objetivo del tema**
>
> En este tema aprenderás a desarrollar Apps con interfaces más sofisticadas utilizando Fragments.

5.1 Fragments

Una App simple, como las que hemos estado creando hasta ahora, tienen un único layout que ocupa toda la superficie de la pantalla del dispositivo. Hemos visto también que, si tenemos que hacer Apps para varios dispositivos muy diferentes (móviles, tablets, etc.…) podemos realizar diferentes layouts para cada tamaño de pantalla. Pero esto, no siempre es suficiente.

Un **Fragment** en Android es un componente reutilizable de interfaz que viene a solucionar este problema. No es una "pantalla completa" con un simple Layout en una Actividad como estábamos acostumbrados, sino **una parte** de la UI con su propio ciclo de vida, sus vistas y su comportamiento. Dicho de otro modo, nos permite dividir una interfaz compleja en piezas más pequeñas, manejables y desacopladas, algo fundamental en aplicaciones modernas.

Por ejemplo: una App diseñada para móviles nos muestra un listado con el contenido de nuestro buzón de correo. Dado que en esa pantalla no tenemos espacio, si queremos ver un correo, lo que sucede es que se cambia a otra pantalla y listo. Pero esa misma App en una tablet no tendría ese problema. Podríamos mostrar a un lado el listado y en el otro, el correo.

Lo que podríamos hacer es modularizar la App en **dos fragmentos** separados. Uno lista los correos, el otro muestra el detalle de un correo seleccionado. El tamaño de la pantalla será el que finalmente condicione si se van a mostrar ambos fragmentos a la vez o no.

Los **Fragments** se utilizan sobre todo para modularizar la interfaz, facilitar la navegación dentro de una única Activity y seguir arquitecturas modernas (como el patrón Single Activity). Esto permite cambiar secciones de la pantalla dinámicamente sin tener que lanzar nuevas Actividades.

5.2 Creación de Fragments

Los Fragments, al igual que las actividades, están compuestos de una clase (que en este caso hereda de Fragment) y de un Layout. Dispone de los mismos métodos del Ciclo de Vida que una Actividad; pero en este caso tenemos que definir un método adicional **onCreateView ().**

A diferencia de una Actividad, no tendremos disponibles los métodos **findViewById ()** y similares de la clase R, ni tendremos disponible el Contexto.

Otra diferencia es que los Fragment no tienen que definirse en el AndroidManifest ya que no son actividades, sino componentes de estas. Para definir un Fragment solamente habría que añadir la clase específica y sus layouts correspondientes.

5.2 Creación dinámica de Fragments

Veamos un ejemplo sencillo. Una App que lista una serie de mensajes de correo y muestra luego su detalle. Vamos a hacer que la asignación de cada Fragment al Layout se haga **en tiempo de ejecución**.

Paso 1 – Crea el proyecto: no es necesario hacer nada diferente a lo que hemos estado haciendo hasta ahora.

Paso 2 – Layout: Abrimos el layout de nuestra actividad y lo editamos en modo texto. Dado que con los Fragments los layouts de la actividad sólo funcionan como contenedores, no hace falta que contengan nada.

```
<FrameLayout
android:id="@+id/fragment_container"
android:layout_width="match_parent"
android:layout_height="match_parent" />
```

Paso 2 – EmailListFragment: Creamos el primer Fragment pulsando New > Fragment > Fragment (Blank). Le damos el nombre de **EmailListFragment**. Veremos que nos genera dos ficheros:

- **EmailListFragment.kt** – en la carpeta del código fuente, donde la MainActivity. Veremos que la clase tendrá un montón de código que luego tendremos que retocar.

- **fragment_email_list.xml** – en la carpeta layout junto al activity_main.xml

Paso 3 – La interfaz 1: Editamos el **fragment_email_list.xml**. Para nuestro ejemplo, este Fragment sólo debe tener un sencillo listado. Para eso, bastaría con añadirle:

```
<ListView
    android:id="@+id/listViewEmails"
    android:layout_width="match_parent"
    android:layout_height="match_parent" />
```

Paso 4 – EmailDetailFragment: Creamos el segundo Fragment y le llamamos **EmailDetailFragment**. Veremos que nos genera dos ficheros:

- **EmailDetailFragment.kt** – en la carpeta del código fuente.

- **fragment_email_detail.xml** – en la carpeta layout junto al activity_main.xml

Paso 5 – La interfaz 2: Editamos el **fragment_email_detail.xml**. Para nuestro ejemplo, este Fragment sólo debe tener un sencillo listado. Para eso, bastaría con añadirle:

```
<TextView
    android:id="@+id/textViewEmail"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:textSize="24sp" />
```

Paso 6 – Lógica del Fragmento 1: Vamos a añadir la lógica del primer Fragment. Abrimos la clase **EmailListFragment.kt** y dejemos en código de la siguiente forma.

```
class EmailListFragment : Fragment() {

    private val emails = listOf(
        "Correo 1",
        "Correo 2",
        "Correo 3"
    )

    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        // Inflate the layout for this fragment
        val view =  inflater.inflate(R.layout.fragment_email_list, container, false)

        val listView = view.findViewById<ListView>(R.id.listViewEmails)

        val adapter = ArrayAdapter(
            requireContext(),
            android.R.layout.simple_list_item_1,
            emails
        )

        listView.adapter = adapter

        listView.setOnItemClickListener { _, _, position, _ ->
            val selectedEmail = emails[position]

            val fragment = EmailDetailFragment()

            val bundle = Bundle()
            bundle.putString("email", selectedEmail)
            fragment.arguments = bundle

            parentFragmentManager.beginTransaction()
                .replace(R.id.fragment_container, fragment)
                .addToBackStack(null)
                .commit()
        }

        return view
    }
}
```

Lo que hace esta clase es cargar en un ListView la lista de emails. Adicionalmente, cada elemento de la lista tiene un evento que, al ser disparado, envía un **Bundle** con información necesaria para el fragmento 2.

Fíjate que estamos llamando a **replace ()**. Este método reemplaza un Fragment existente en un contenedor. A efectos, es como si estuviésemos reemplazando un Fragment por otro.

Paso 7 – Lógica del Fragmento 2: Vamos a añadir la lógica del segundo Fragment. En este caso, simplemente mostraremos el contenido en un textView.

```
class EmailDetailFragment : Fragment() {

    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        val view = inflater.inflate(R.layout.fragment_email_detail, container, false)

        val textView = view.findViewById<TextView>(R.id.textViewEmail)

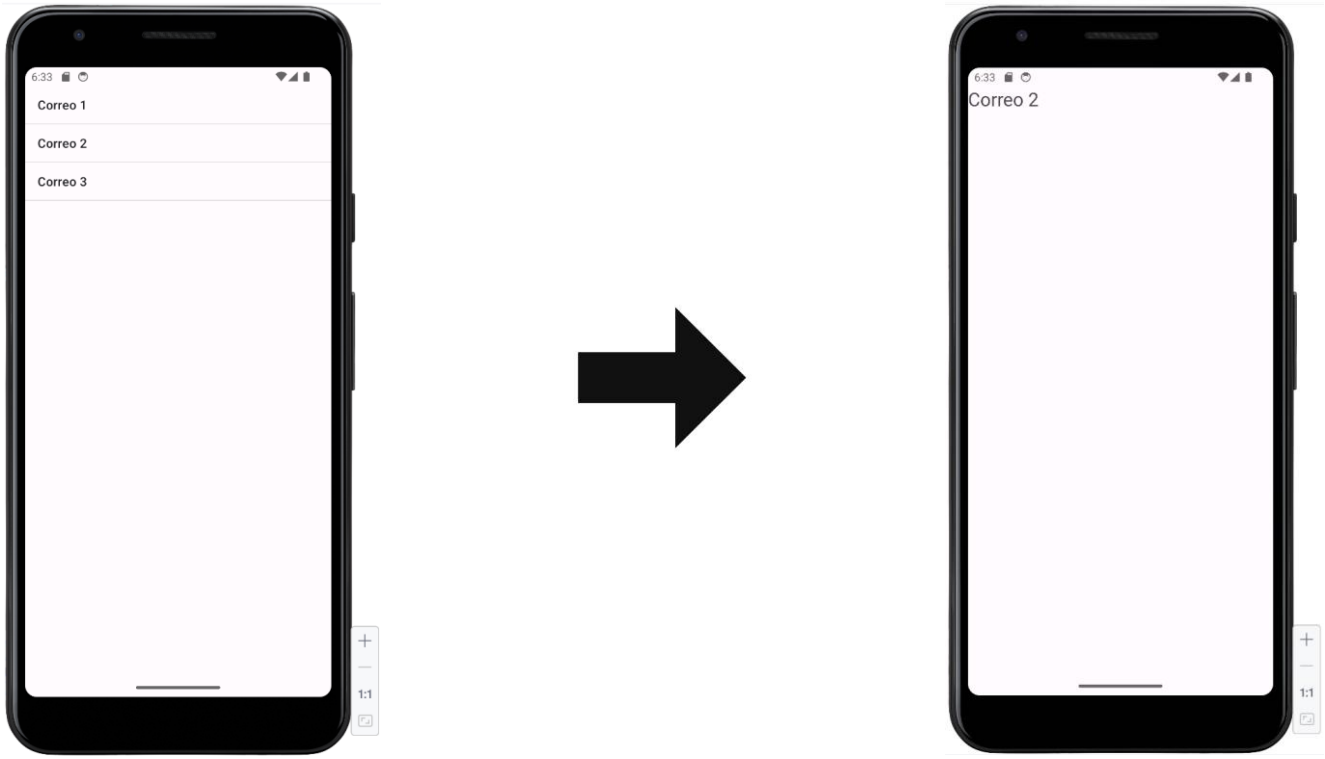
        val email = arguments?.getString("email")
        textView.text = email

        return view
    }
}
```

Paso 8 – Cargar el primer Fragmento: Nos falta indicar en el **MainActivity.kt** cuál de los dos Fragments es el primero que se tiene que mostrar. Para ello, añadimos en esta clase:

```
if (savedInstanceState == null) {
    supportFragmentManager.beginTransaction()
        .replace(R.id.fragment_container, EmailListFragment())
        .commit()
}
```

El resultado visualmente queda similar al de una transición de una actividad a otra:



Paso 9 – Modificar la interfaz 2: Editamos el **fragment_email_detail.xml** para darle algo más de funcionalidad a la App. En concreto, queremos 'volver' del Fragment 2 al Fragment 1 mediante un botón. Modifica el fichero para usar un LinearLayout, y añade un botón de la siguiente forma:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:padding="16dp"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:id="@+id/textViewEmail"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:textSize="24sp"
        android:text="Email detalle" />

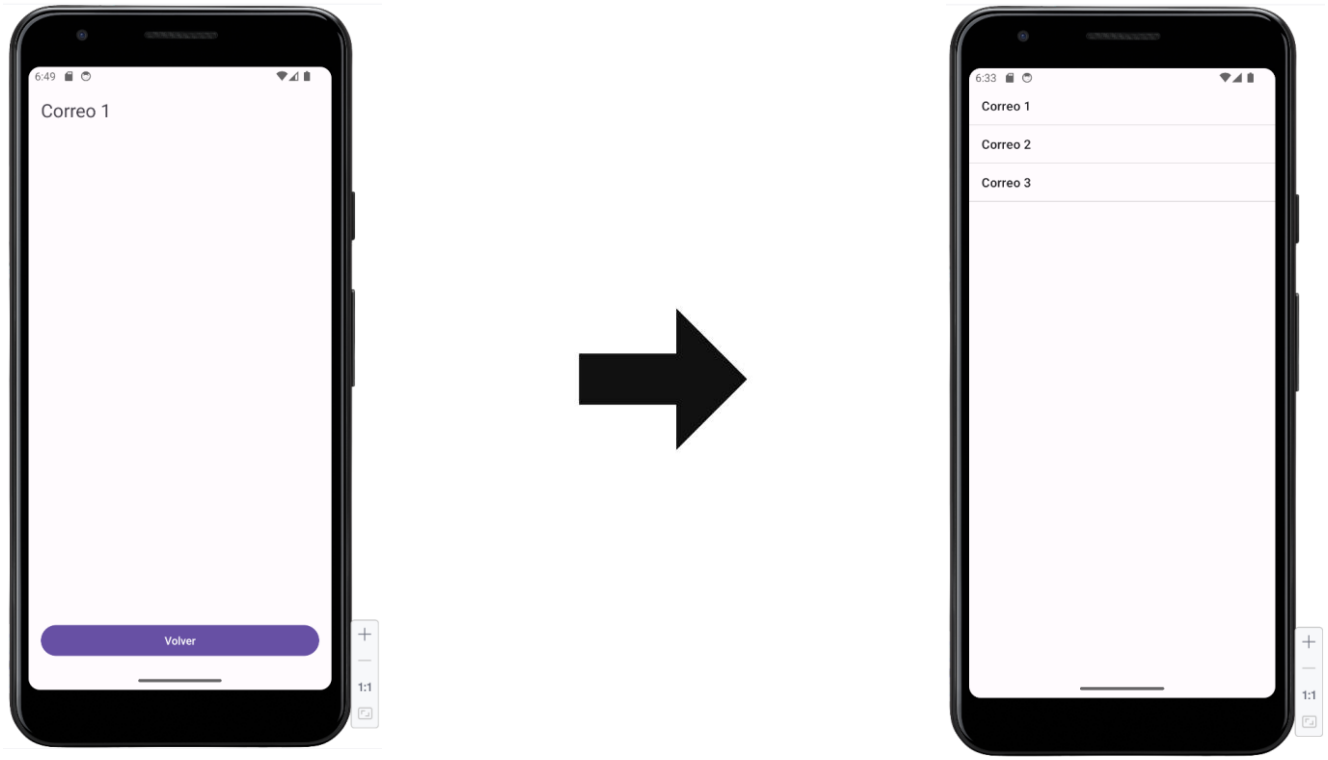
    <Button
        android:id="@+id/buttonBack"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Volver" />
</LinearLayout>
```

Paso 10 – Modificar la lógica del Fragmento 2: Editamos **EmailDetailFragment.kt** para añadirle el **replace()** que nos devolverá al Fragment 1:

```
val buttonBack = view.findViewById<Button>(R.id.buttonBack)
buttonBack.setOnClickListener {

    parentFragmentManager.beginTransaction()
        .replace(R.id.fragment_container, EmailListFragment())
        .commit()
}
```

El resultado:



5.3 Creación estática de Fragments

En este caso, la asignación de los Fragmentos al layout se hacen directamente sobre el **activity_main.xml**, lo que 'fija' los Fragments que podrán mostrarse en la App.

Paso 1 – Proyecto y Fragments: Crea un proyecto nuevo y añade dos Fragments con los mismos nombres que en el ejemplo anterior.

Paso 2 – Añade los Fragments: Bastaría con hacer en el **activity_main.xml** lo siguiente:

```
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="horizontal"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <fragment
        android:id="@+id/fragmentList"
        android:name="es.elorrieta.app.fragments.EmailListFragment"
        android:layout_width="0dp"
        android:layout_height="match_parent"
        android:layout_weight="1" />
    <fragment
        android:id="@+id/fragmentDetail"
        android:name="es.elorrieta.app.fragments.EmailDetailFragment"
        android:layout_width="0dp"
        android:layout_height="match_parent"
        android:layout_weight="1" />

</LinearLayout>
```

Nótese que, si hacemos esto, ambos Fragment existen **SIEMPRE** en nuestra App. Se mostrarán uno al lado del otro. No hay ni navegación ni reemplazo.

Paso 3 – La interfaz 1: Edita **fragment_email_list.xml**. Para que te quede igual que el ejemplo anterior.

Paso 4 – La interfaz 2: Edita **fragment_email_detail.xml**. Para que te quede igual que el ejemplo anterior, pero remueve el button.

Paso 5 – Lógica del Fragmento 1: Igual que la del ejemplo anterior, pero elimina tanto el Bundle como el **beginTransaction ()**. En su lugar, añade:

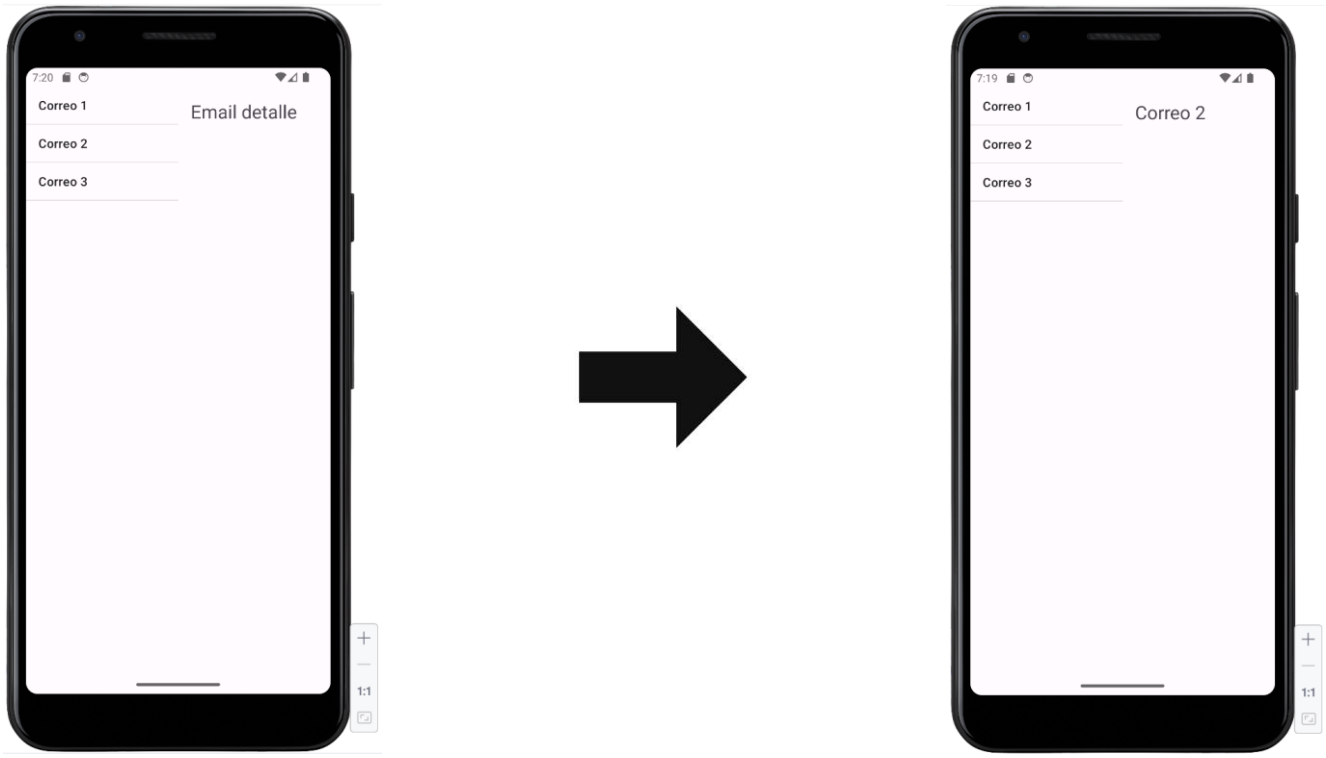
```
val detailFragment =
    parentFragmentManager.findFragmentById(R.id.fragmentDetail) as EmailDetailFragment

detailFragment.mostrarEmail(email)
```

Paso 6 – Lógica del Fragmento 2: Igual que la del ejemplo anterior, pero elimina el botón. Añade el siguiente método siguiente:

```
fun mostrarEmail(email: String) {
    textView.text = email
}
```

El resultado es



Ejercicio 15

Crea una App para gestionar un listado de películas. La App estará estructurada de la siguiente forma:

- Fragment 1 – Mostrará un listado de títulos de películas (RecyclerView). Habrá dos controles en cada elemento de la lista: un botón de 'Detalle' y un icono de corazón (gris). Finalmente, en este Fragment deberá haber otro botón a parte de 'Ver Favoritos'

    - Pulsar 'Detalle' nos lleva al Fragment 2
    - Pulsar 'Corazón' le hace cambiar de color (gris a rojo) y la película se marca como favorita. Seleccionarlo de nuevo la hace cambiar de nuevo (rojo a gris) y se desmarca como favorita.
    - Pulsar 'Ver Favoritos' nos lleva al Fragment 3

- Fragment 2 – Nos mostrará en una serie de campos la información de la película: director, fecha, … Ninguno de estos campos es editable.

- Fragment 3 – Nos mostrará un listado (RecyclerView) con todos los títulos de películas marcadas como favoritas.

Añade los controles de navegación que consideres necesarios para cambiar de un Fragment a otro, así como para ir pasando la información que necesites de uno a otro.

5.3 Transiciones entre Fragments (Dinámicos)

Una **transition** (transición) es el efecto visual que ocurre cuando cambias de un estado a otro en la interfaz. En el caso de los Fragments, es cómo se pasa de una 'pantalla' a la otra 'pantalla' (Fragments). Es principalmente un efecto visual que controla cómo aparece y desaparece un Fragment durante la navegación.

Ten en cuenta que esto NO es una animación.

Existen varios tipos de Transiciones. Para añadir una, simplemente indícaselo mediante setTranssition. En nuestro ejemplo, la clase **EmailListFragment.kt** nos quedaría así:

```
parentFragmentManager.beginTransaction()
    .setTransition(FragmentTransaction.TRANSIT_FRAGMENT_CLOSE)
    .replace(R.id.fragment_container, fragment)
    .addToBackStack(null)
    .commit()
```